EXP 6	Workflow

Aim:

To create and configure workflows in Zoho Creator for automating validations, calculations and room status updates in the Hotel Room Booking application.

Theory:

Workflows automate actions on form events: on load, on user input, on validation and on successful submission. Four workflows are configured on the Room Booking Form:

- **Validate Booking** (on validation) - Check-Out after Check-In, guests within room capacity, no date clash for the same room.
- **Fill Room Details** (on user input of Room) - fills Room Type and Rate and calculates the bill.
- **Calculate Nights and Amount** (on user input of Check-Out Date).
- **Update Bill and Room Status** (on successful submission) - stores the bill and sets Room Status to Occupied on Check-In and Available on Check-Out.

Procedure:

1. Open the Workflow section and click New Workflow.
2. Choose Room Booking Form, Created or Edited, Validations on form submission and name it Validate Booking.
3. Add a Deluge script action with the validation logic and save.
4. Create Fill Room Details (User input of Room) and Update Bill and Room Status (Successful form submission).
5. Verify that all workflows are enabled.
6. Submit the test cases from the sample data and verify the alerts.

Deluge - Validate Booking (Validations on form submission):

```
// Validation 1: Check-Out Date must be after Check-In Date
if(input.Check_In_Date != null && input.Check_Out_Date != null && input.Check_Out_Date
<= input.Check_In_Date)
{
    alert "Check-Out must be after Check-In. Submit cancelled.";
    cancel submit;
}
if(input.Room != null)
{
    rm = Rooms[ID == input.Room];
    // Validation 2: guests between 1 and room capacity
    if(input.Number_of_Guests == null || input.Number_of_Guests < 1 ||
input.Number_of_Guests > rm.Capacity)
    {
        alert "Guests must be between 1 and the room capacity (" + rm.Capacity + ").";
        cancel submit;
    }
    // Validation 3: the room must not be booked for overlapping dates
    if(input.Status != "Checked-Out" && input.Check_In_Date != null &&
input.Check_Out_Date != null)
    {
        clash = Room_Booking_Form[Room == input.Room && Status != "Checked-Out" &&
```

```
Check_In_Date < input.Check_Out_Date && Check_Out_Date > input.Check_In_Date && ID !=
input.ID];
        if(clash.count() > 0)
        {
            alert "Room " + rm.Room_Number + " is already booked for these dates
(Booking ID " + clash.Booking_ID + ").";
            cancel submit;
        }
    }
}
```

Deluge - Update Bill and Room Status (Successful form submission):

```
// Server-side calculation of room details and bill
rm = Rooms[ID == input.Room];
nights = input.Check_In_Date.daysBetween(input.Check_Out_Date);
input.Room_Type = rm.Room_Type;
input.Rate_Per_Night = rm.Rate_Per_Night;
input.Nights = nights;
input.Total_amount = nights * rm.Rate_Per_Night;
// Keep the room status in sync with the booking status
if(input.Status == "Checked-In")
{
    rm.Room_Status = "Occupied";
}
else if(input.Status == "Checked-Out")
{
    rm.Room_Status = "Available";
}
```

Output:

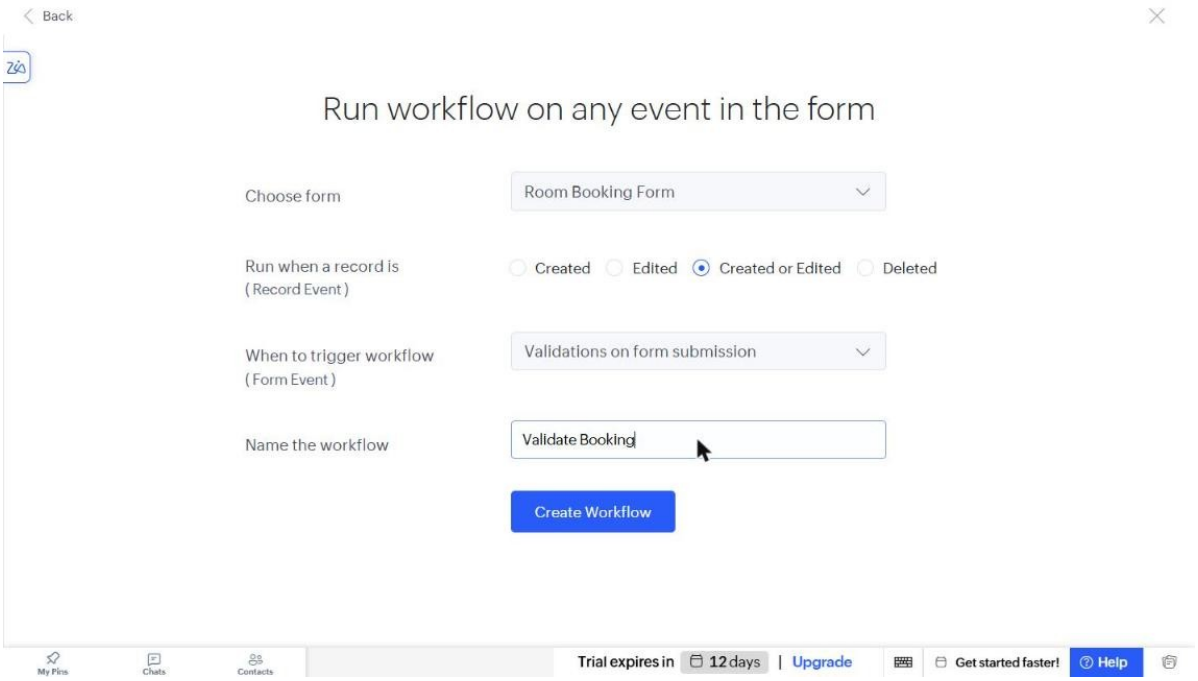


Fig. 6.1: Creating the Validate Booking workflow

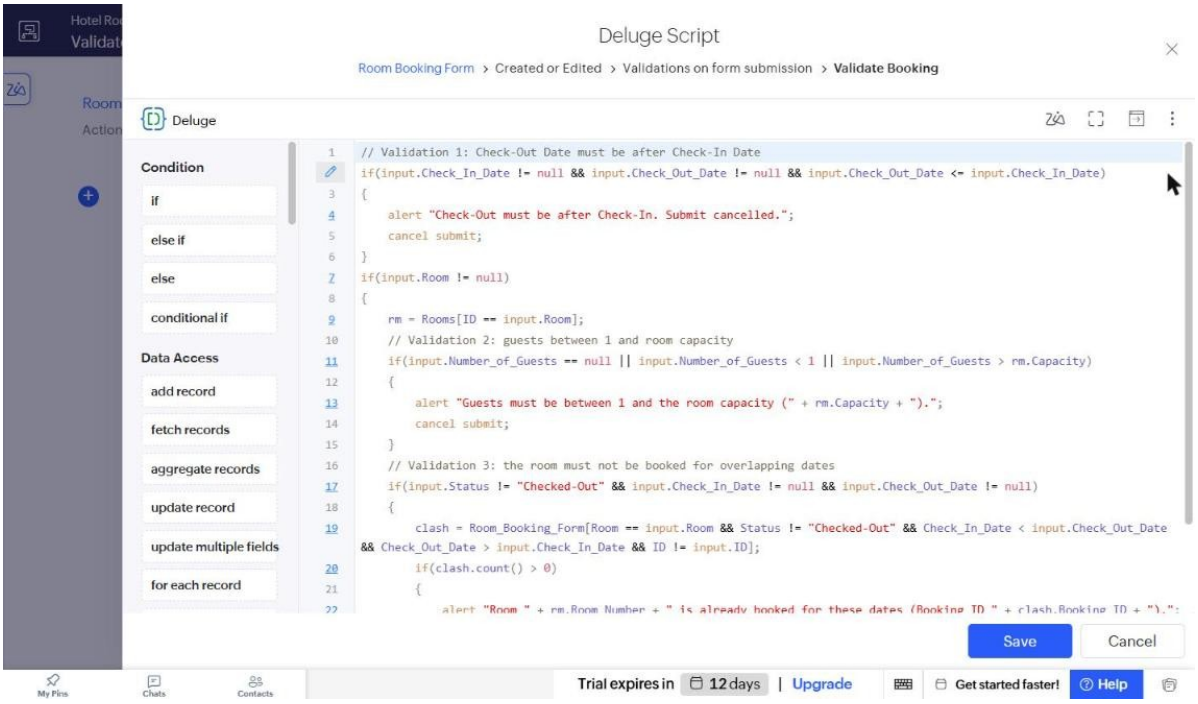


Fig. 6.2: Validation script

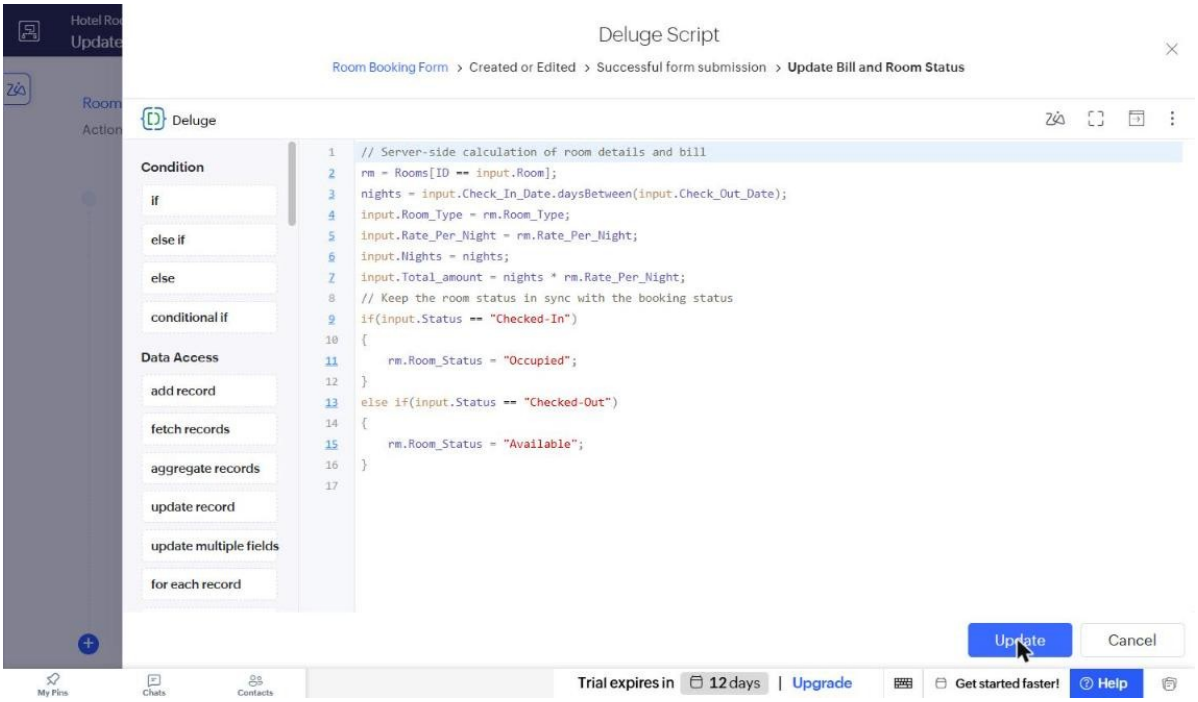


Fig. 6.3: Bill and room status update script

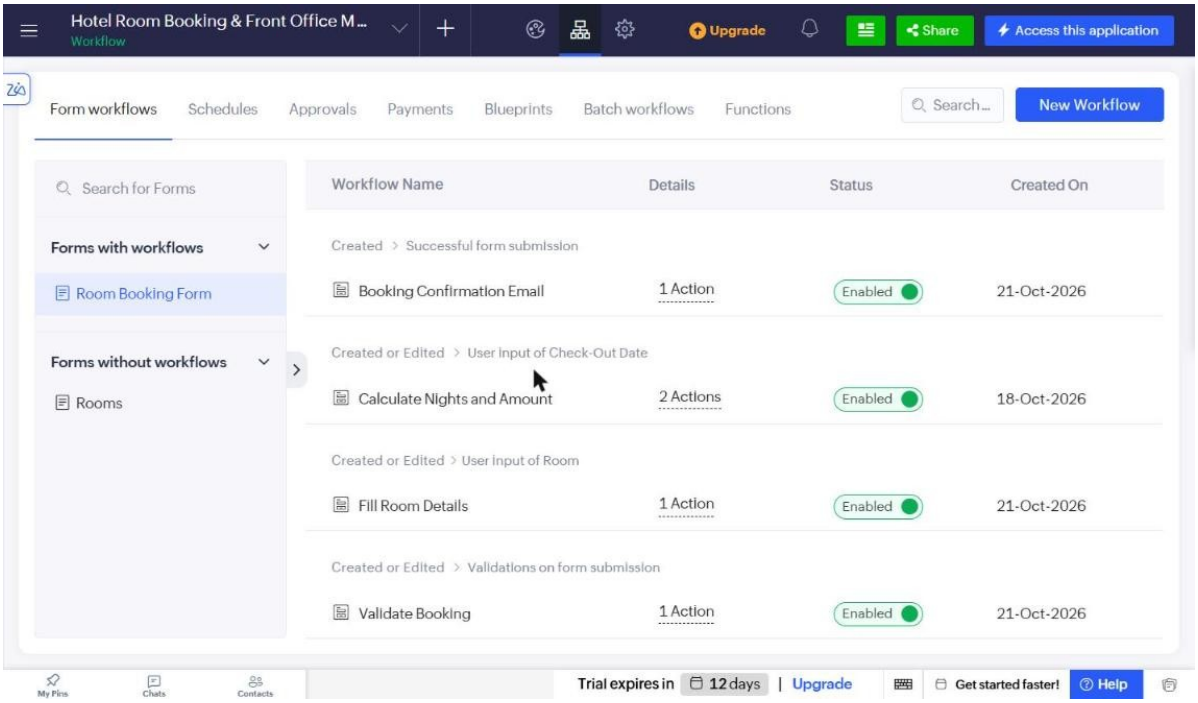


Fig. 6.4: All workflows saved and enabled

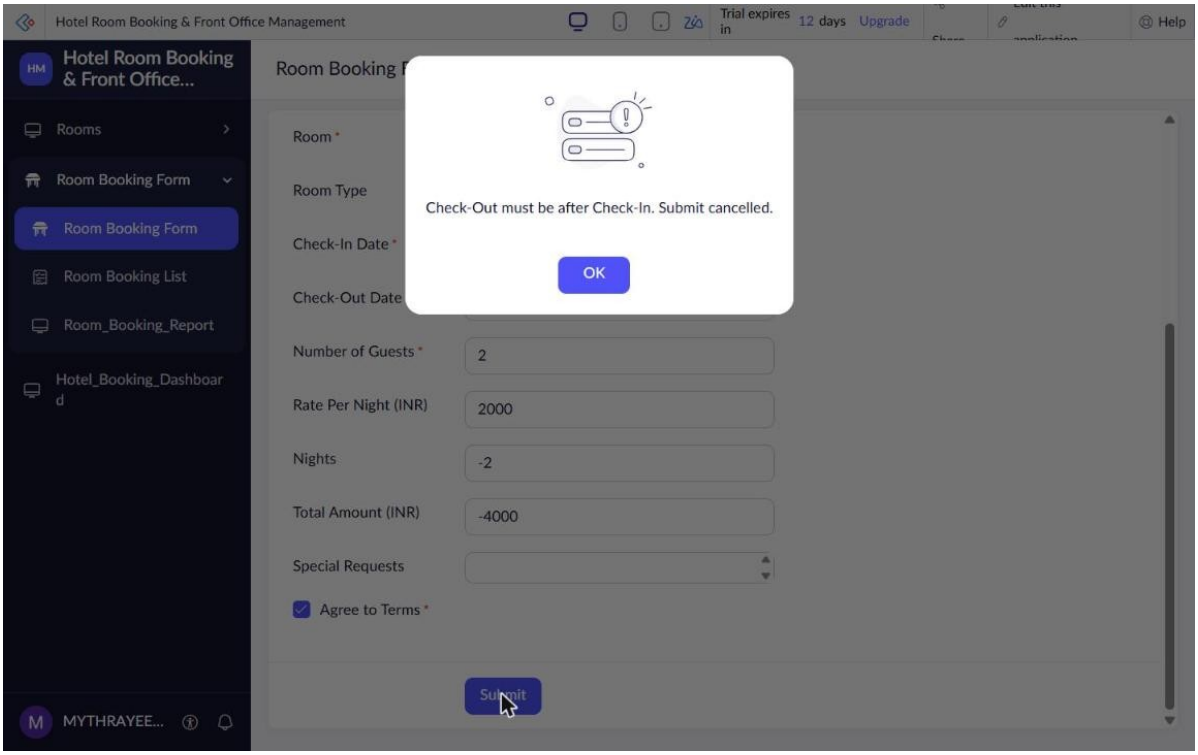


Fig. 6.5: Test 1 - Check-Out before Check-In blocked

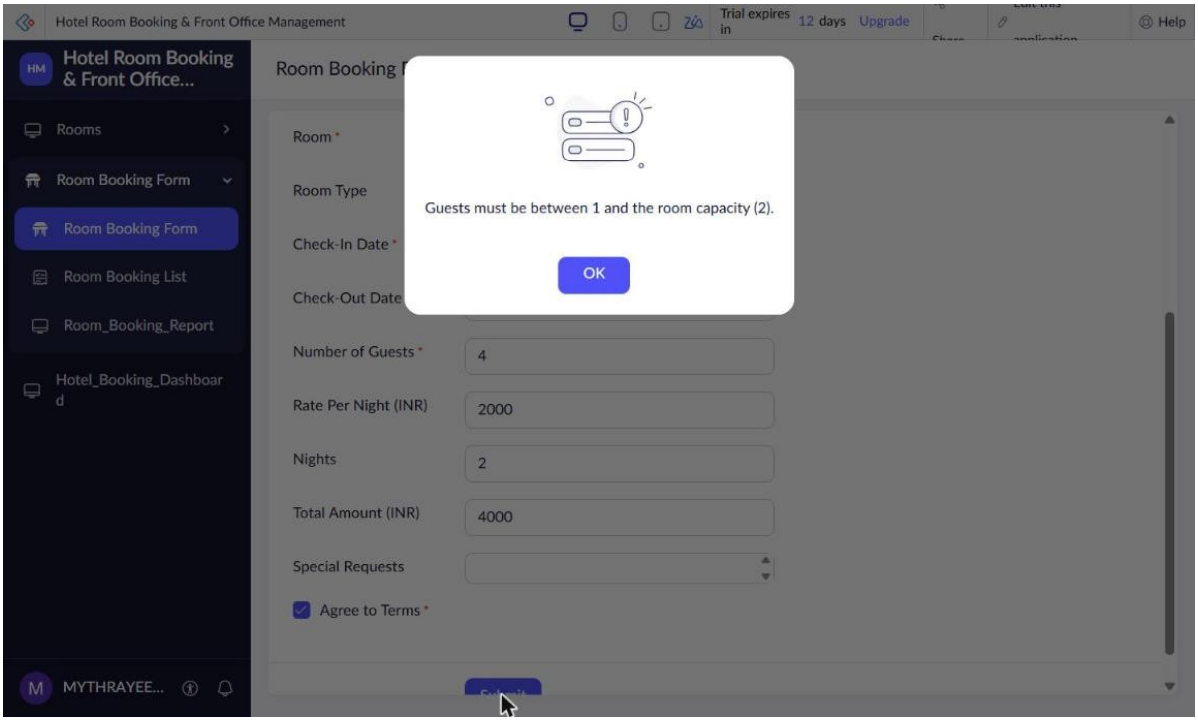


Fig. 6.6: Test 2 - 4 guests for room 101 (capacity 2) blocked

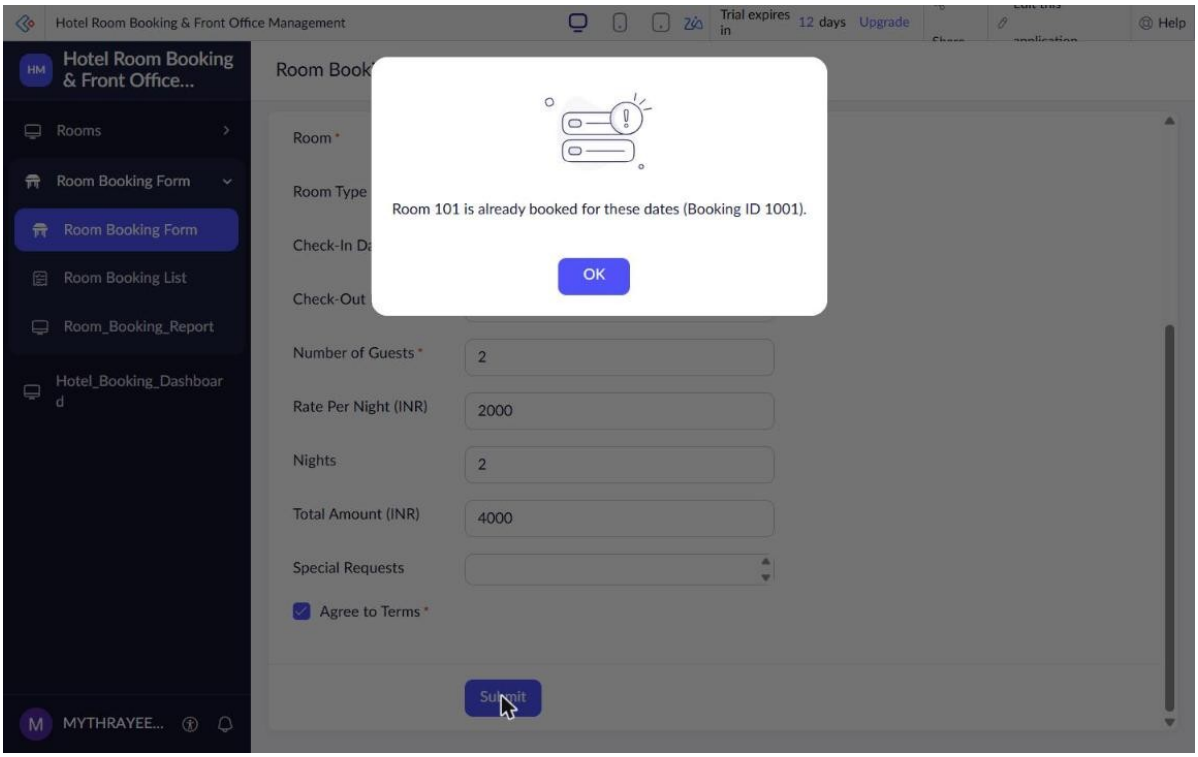


Fig. 6.7: Test 3 - date clash with Booking 1001 blocked

Observation:

Test Case	Input	Result
Check-Out before Check-In	101, 05-Nov to 03-Nov	Alert shown, submit cancelled

Too many guests	101, 4 guests	"Guests must be between 1 and the room capacity (2)"
Date clash	101, 21-Oct to 23-Oct	"Room 101 is already booked (Booking ID 1001)"
Mandatory field empty	Phone empty	"This field is mandatory"

Result:

Workflows were created and enabled for validation, automatic calculation and room status update. All test cases from the sample data produced the expected results.